

Отчёт по лабораторной работе №6

Дисциплина: Архитектура компьютеров

Мухина Ксения Николаевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Задания для самостоятельной работы	12
4	Выводы	15
	Список литературы	16

Список иллюстраций

2.1	Создание каталога и файла для работы.	6
2.2	Программа lab6-1.	6
2.3	Изменение программы lab6-1.	7
2.4	Изменённая программа lab6-1.	7
2.5	Программа lab6-2.	7
2.6	Изменение программы lab6-2.	8
2.7	Изменённая программа lab6-2.	8
2.8	Изменённая программа lab6-2.	8
2.9	Программа lab6-3.	8
2.10	Изменение программы lab6-3.	9
2.11	Изменённая программа lab6-3.	9
2.12	Программа variant.	10
3.1	Программа variant12.	14

Список таблиц

1 Цель работы

Освоить арифметические инструкции языка ассемблера NASM.

2 Выполнение лабораторной работы

Далее описываемая работа была выполнена на виртуальной машине Oracle VirtualBox с ОС Linux Ubuntu.

1. Символьные и численные данные в NASM

Перед началом работы создадим каталог lab06, перейдём в него и создадим файл lab6-1.asm.

```
knmuhina@virtualbox:~$ mkdir ~/work/arch-pc/lab06
knmuhina@virtualbox:~$ cd ~/work/arch-pc/lab06
knmuhina@virtualbox:~/work/arch-pc/lab06$ touch lab6-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.1: Создание каталога и файла для работы.

Введём в файл текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-1
j
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.2: Программа lab6-1.

Результат программы - „j“, так как программа складывает двоичные коды символов „6“ и „4“ (00110110 + 00110100), в результате выдавая сумму 01101010 (106), то есть код символа j.

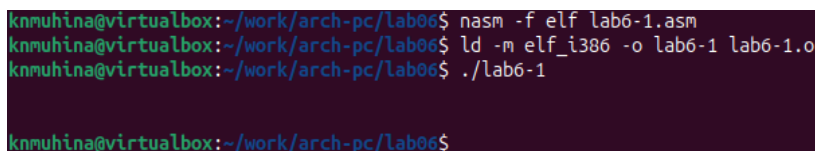
Изменим некоторые строки программы следующим образом:



The diagram shows two columns of assembly code separated by a large right-pointing arrow. The left column contains the instructions `mov eax, '6'` and `mov ebx, '4'`. The right column contains the instructions `mov eax, 6` and `mov ebx, 4`.

Рисунок 2.3: Изменение программы lab6-1.

Теперь запустим программу.



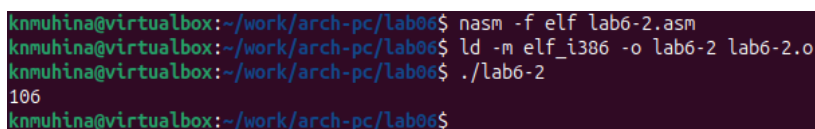
```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-1.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-1 lab6-1.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-1
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.4: Изменённая программа lab6-1.

В данном случае результатом программы является код 10, который принадлежит символу перевода строки. Вследствие, при работе программы мы не видим никаких символов.

Чтобы программа корректно работала с числами, в файле `in_out.asm` реализованы подпрограммы для преобразования ASCII символов в числа и наоборот. Используем эти функции для следующей программы.

Создадим файл `lab6-2.asm`, в который введём текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.



```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-2
106
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.5: Программа lab6-2.

В результате мы получим число 106. В данном случае команда `add` складывает десятичные коды символов „6“ и „4“ ($54 + 52$). В отличие от предыдущей программы, функция `iprintLF` позволила вывести само число, а не символ, кодом которого оно является.

Изменим некоторые строки программы следующим образом:

Рисунок 2.6: Изменение программы lab6-2.

Теперь запустим программу.

```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-2
10
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.7: Изменённая программа lab6-2.

Теперь результатом является число 10, то есть сумма чисел 6 и 4.

Заменяем функцию `iprintLF` на `iprint`:

```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-2.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-2 lab6-2.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-2
10knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.8: Изменённая программа lab6-2.

Результат вычислений является таким же, однако изменённая функция не включает в себя перевод строки. По этой причине поле ввода команды появилось на той же строке, что и результат программы.

2. Выполнение арифметических операций в NASM

В качестве примера выполнения арифметических операций приведём программу вычисления остатка в выражении $(5 \cdot 2 + 3) / 3$.

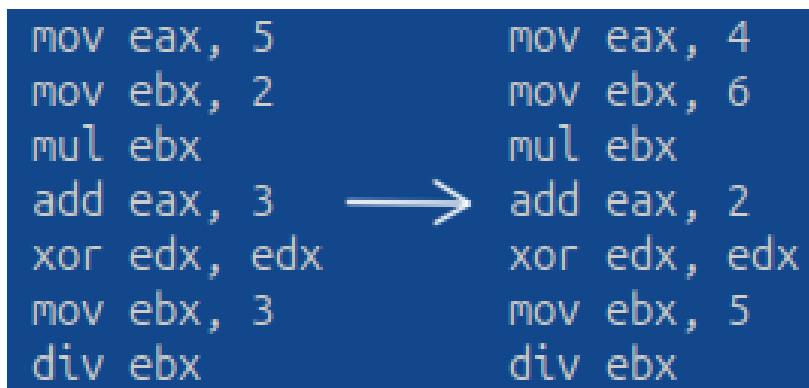
Создадим файл `lab6-3.asm`, в который введём текст программы из листинга, представленного в лабораторной работе, и создадим исполняемый файл.

```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-3
Результат: 4
Остаток от деления: 1
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.9: Программа lab6-3.

Программа сработала корректно и выдала верный результат.

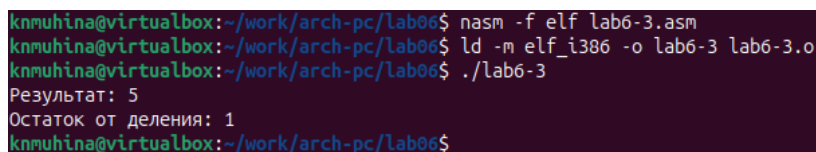
Изменим программу так, чтобы она выводила остаток, получившийся в результате вычисления выражения $(4*6 + 2)/5$. Изменим некоторые строки программы следующим образом:



<code>mov eax, 5</code>	→	<code>mov eax, 4</code>
<code>mov ebx, 2</code>		<code>mov ebx, 6</code>
<code>mul ebx</code>		<code>mul ebx</code>
<code>add eax, 3</code>		<code>add eax, 2</code>
<code>xor edx, edx</code>		<code>xor edx, edx</code>
<code>mov ebx, 3</code>		<code>mov ebx, 5</code>
<code>div ebx</code>		<code>div ebx</code>

Рисунок 2.10: Изменение программы lab6-3.

Теперь запустим программу.



```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf lab6-3.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o lab6-3 lab6-3.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./lab6-3
Результат: 5
Остаток от деления: 1
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 2.11: Изменённая программа lab6-3.

Программа выдала верный результат.

В качестве другого примера рассмотрим программу вычисления варианта задания по номеру студенческого билета. Создадим файл `variant.asm`, введём в него текст программы из листинга, представленного в лабораторной работе.

Создадим исполняемый файл и запустим программу.

```

knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf variant.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant variant.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./variant
Введите № студенческого билета:
1032253531
Ваш вариант: 12
knmuhina@virtualbox:~/work/arch-pc/lab06$

```

Рисунок 2.12: Программа variant.

Программа выдала верный результат. В ходе вычислений программа разделила число (номер билета) на 20, получив остаток 11. Далее она прибавила к получившемуся остатку 1, и итоговое число вывела как результат работы программы.

Разберём подробнее листинг программы.

В приведённом в лабораторной работе листинге строки

```

mov ecx, x
mov edx, 80
call sread

```

используются для ввода номера студенческого билета с клавиатуры и его записи для последующей работы с номером.

Инструкция „call atoi“ используется для перевода введённого номера, представленного ASCII символами, в число, которое будет использовано для вычисления варианта. Число будет записано в регистр ecx как будущее делимое.

Строки

```

xor edx, edx ; обнуление для корректной работы div.
mov ebx, 20 ; запись делителя в ebx.
div ebx ; деление номера на 20. Остаток от деления записывается в регистр edx.
inc edx ; увеличение остатка от деления на 1.

```

отвечают за само вычисление варианта. В качестве комментариев даны пояснения к каждой инструкции.

Строки

```
mov eax, rem  
call sprint
```

отвечают за вывод на экран сообщения „Ваш вариант:“.

Строки

```
mov eax, edx  
call iprintLF
```

отвечают за вывод на экран результата вычислений.

3 Задания для самостоятельной работы

Напишем программу для вычисления выражения $(8x - 6)/2$. Мы выбрали именно данное выражение, так как оно является 12 вариантом задания.

Листинг 1. Программа `variant12.asm`

```
%include 'in_out.asm'

SECTION .data
var: DB 'Вычисление (8x - 6)/2', 0
msg: DB 'Введите x: ', 0
res: DB 'Результат: ', 0

SECTION .bss
x: RESB 80

SECTION .text
GLOBAL _start
_start:

mov eax, var
call sprintLF
```

```
mov eax, msg  
call sprint
```

```
mov ecx, x  
mov edx, 80  
call sread
```

```
mov eax, x  
call atoi
```

```
mov ebx, 8  
mul ebx  
sub eax, 6
```

```
xor edx, edx  
mov ebx, 2  
div ebx  
mov ebx, eax
```

```
mov eax, res  
call sprint  
mov eax, ebx  
call iprintLF
```

```
call quit
```

Проверим работу программы для значений 1 и 5.

```
knmuhina@virtualbox:~/work/arch-pc/lab06$ nasm -f elf variant12.asm
knmuhina@virtualbox:~/work/arch-pc/lab06$ ld -m elf_i386 -o variant12 variant12.o
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./variant12
Вычисление (8x - 6)/2
Введите x: 1
Результат: 1
knmuhina@virtualbox:~/work/arch-pc/lab06$ ./variant12
Вычисление (8x - 6)/2
Введите x: 5
Результат: 17
knmuhina@virtualbox:~/work/arch-pc/lab06$
```

Рисунок 3.1: Программа variant12.

Программа сработала корректно и выдала корректный результат.

4 Выводы

В ходе выполнения лабораторной работы мы освоили арифметические инструкции языка ассемблера NASM.

Список литературы

1. Файл «Лабораторная работа №6. Арифметические операции в NASM.pdf»